deCDN Litepaper

Alper Gundogdu · Yigit Gurbulak · Altan Oruc · Ant Somers

deCDN Labs

2026-05

Abstract

deCDN clears content delivery at approximately **$0.01 per gigabyte** on a posted-price basis, settled per megabyte in USDC, with no minimum commitment — roughly an order of magnitude below the rates incumbent CDNs lock into year-long contracts. The pivot is a market-design one. Capacity, bandwidth, and interconnect density have grown faster than demand for a decade; what has not scaled is the supply side. deCDN opens it: any operator with bandwidth and a capacity bond can register and serve, every operator posts their own rate, and clients pick on price, latency, and reputation. Bytes are addressed by their BLAKE3 content hash and verified chunk-by-chunk on arrival. Origins stay hidden. Settlement happens off-chain over reusable USDC payment pools; a node redeems its cumulative voucher on-chain in a single transaction, and the funder reclaims whatever is unspent after a grace window.

This paper covers the protocol primitives, the platform architecture, the on-chain trust surface, the token and governance model, and the trade-offs the design makes.

This document is informational only and does not constitute an offer to sell or a solicitation to buy any security or token. Technical and economic parameters described herein are provisional and subject to change. TOKEN is a utility and work-token bonding asset; governance rights accrue only to operators who bond TOKEN and serve bytes, never to passive holders. It is not offered or sold as a security, and the protocol does not promise any return on holding. Distribution of this document is restricted in jurisdictions where it would be unlawful.

1. Introduction

The week Meta released Llama 3, HuggingFace slowed to a crawl. The cause was structural, not transient: data volumes are compounding across every category that matters — AI model weights, scientific datasets, software packages, high-resolution media — while the infrastructure that delivers them is concentrated in a small set of providers built for serving webpages, not multi-gigabyte files. Popular releases overwhelm the gates. Small teams burn five-digit monthly bills on a single viral moment. Larger teams pre-provision for peaks that arrive twice a year and pay for ghost capacity the other 363 days.

That is a market-design problem, not a capacity problem. Wholesale bandwidth has fallen substantially over the past decade; commercial CDN rates have not. Supply is concentrated in a small set of incumbent providers that sell on year-long contracts with monthly minimums and price opaquely behind a sales relationship. Capacity exists in aggregate; the gates between it and the people who want it are narrow, expensively held, and provider-discretionary.

deCDN's design follows from that diagnosis. Five principles run through the protocol:

- **Content-addressed.** Every blob is identified by the BLAKE3 hash of its bytes. No URLs, no domain names, no URL redirects — the hash is the only identifier that matters, and any operator that has cached it can serve it.
- **Demand-shaped.** The delivery graph follows what clients ask for, not what was provisioned in last year's procurement cycle. Capacity expands per request, retracts when demand fades.
- **Stable-currency settlement.** Operators are paid in USDC the moment bytes flow. TOKEN is reserved for bonding and governance. Operator P&L is predictable; clients are not exposed to token price.
- **Publisher-fundable.** Delivery is paid the way CDN egress is paid today. A `PaymentPool` separates the funder from the voucher signer, so a publisher or app can fund its users' downloads — the funder owns the pool, delegated keys sign the vouchers, and the publisher's bandwidth budget pays operators per megabyte instead of paying egress to a single cloud, so the end user needs no token balance, no gas, and no on-chain transaction. ERC-4337 account abstraction layers gas abstraction on top of the same funded-pool primitive. The consumer per-megabyte path remains; the payer is no longer forced to be the downloader.
- **Slashing-backed accountability.** Operators bond into a registry with on-chain slashing for the two protocol offenses — rate manipulation and blacklist violations. Corrupted delivery never reaches a slash: it is caught at the wire by the client's BLAKE3 verification before payment is signed, so a tampered byte is simply never paid for. Honest operators earn per-byte revenue; misbehaving ones lose bond to a permissionless, bonded on-chain challenge.

The contrast across the dimensions an investor would weigh:

Dimension	Traditional CDN	deCDN
Infrastructure model	Closed, incumbent-owned PoPs	Permissionless peer mesh
Pricing	Negotiated, opaque ("contact sales")	Posted per operator, per-megabyte
Payment model	Year-long contracts, monthly invoice	Off-chain USDC pool, per-megabyte metering
Who pays	Publisher buys egress; end users free	Publisher sponsors downloads *or* consumer pays per megabyte; sponsored users need no token, gas, or transaction
Single point of failure	One provider per route	Many operators per hash; failed ranges re-dispatch
Scalability	Capacity provisioned in advance	Demand-shaped; supply expands per request
Censorship resistance	Provider-discretionary takedown	Public on-chain blacklist, global and per-region scopes
Origin trust	URL exposed to the network	Backing store hidden; the network sees only content hashes and NodeIds
Operator incentives	Internal compensation	Slashing-backed; bond + per-byte revenue

1.1 Market sizing

Centralised CDNs sell on enterprise contracts with year-long minimums and a sales relationship in front of every quote. The cheapest tier is reserved for very large pre-paid commitments; the everyday tier sits substantially higher. Wholesale transit pricing has fallen meaningfully across the same decade; retail CDN pricing has not, and the gap is rent extracted at the gate rather than an infrastructure cost. deCDN opens that supply side: its posted clearing target sits roughly an order of magnitude below the upper end of incumbent retail, feasible because any operator backed by a zero-egress origin store can clear at market rate without losing money on cache misses, while one paying full retail egress cannot. The operators who can clear set the floor; everyone else has to cache aggressively or price above market.

2. The Engine

The protocol composes four primitives — addressing, transport, discovery, payment. A client computes the BLAKE3 hash of what it wants, looks the hash up in the DHT to find operators holding it, probes the candidates for price and latency within a bounded collection window, opens QUIC streams to the best-scoring few, and signs cumulative USDC vouchers as bytes arrive. The pool that backs those vouchers is opened once and reused across downloads, not closed per fetch; each operator redeems the vouchers it holds on its own schedule. That path — hash to bytes to settlement — is what the four primitives compose, each at a distinct layer of the stack:

Layer	Technology	Responsibility
Network transport	iroh (QUIC, NAT traversal)	Encrypted byte transport, parallel streams
Addressing	BLAKE3 content hash	Identifier; per-chunk in-flight verification
Discovery	Kademlia DHT (`cdn/dht/v1`)	Hash → operator lookup; node set and region resolved from the on-chain registry
Payment	Off-chain USDC payment pool	Per-MiB hash-chain metering; cumulative vouchers redeemed on-chain independently
Settlement	On-chain smart contracts	Pool close, slashing, governance, fee routing
Bond & accountability	`CapacityBond` + `SlashJudge`	Operator skin-in-the-game; signed-evidence slash adjudication
Compliance	`ContentBlacklist` + `PublisherRegistry` + `OriginAssignment`	Publisher vetting and origin seating; on-chain hash takedown

Addressing. Every blob is identified by the BLAKE3 hash of its bytes. A client asks for hash `h`; any operator that has cached `h` can serve it. Verification is symmetric: the client computes the hash of every chunk as it arrives and disconnects on mismatch. Encrypted and unencrypted payloads are different blobs at this layer, and the protocol does not know or care which is which.

Transport. Bytes move over QUIC via iroh, with ALPN-negotiated wire protocols for each role. Every connection completes a full TLS 1.3 handshake before application bytes flow — no early data, no replay risk. Two specifications, both in draft, extend this: bao verified-range streaming lets a client check any byte range independently instead of only whole files, and multi-source fetch uses that to pull a large blob from several operators at once and aggregate their throughput. Mismatched bytes trigger immediate disconnect and re-dispatch to a different source — the corrupted range is never paid for, since vouchers are signed only after verification.

Discovery. Operators publish what they cache to a Kademlia DHT. The node set itself — and each node's self-attested region — comes from the on-chain `CapacityBond` registry, not from any peer-to-peer chatter; the DHT carries content inventory only. Clients look hashes up in the DHT and bootstrap their peer table from that registry, with a locally cached peer list as fallback during a transient RPC outage. No central tracker exists — no single party can rate-limit discovery or withhold routing. Regional proxy warming seeds the first local copy of a blob as a side effect of ordinary delivery: when a client finds only distant holders, it routes its request through a nearby operator instead, which pulls the blob to serve the client and keeps a copy — becoming the first local holder.

Payment. A funder opens one USDC payment pool and reuses it — across nodes, across sessions, across downloads. There is no per-fetch open and no on-chain minimum: the owner sizes a single working deposit, escrows it, and tops it up as it drains. The pool funds capped, delegated signers — the owner's own key, or per-device or per-session keys it issues — each bounded by a spending cap and an expiry, so a delegated key is revocable and never holds funds of its own. Metering rides a hash chain: each megabyte delivered releases one unsigned preimage

that advances the running total, while a signed voucher anchors that total — *I have received X megabytes, here is payment for X × rate* — and one more seals it at close, so signatures stay occasional rather than one per megabyte. The operator verifies each step incrementally. Delivery is not lock-stepped to payment: a bounded credit window lets bytes keep flowing ahead of voucher acknowledgement, so delivery and payment run concurrently instead of payment gating each chunk. Each operator redeems the strongest voucher it holds on-chain at any time — a cumulative claim only ever grows, so redemption needs no dispute round — and the owner closes the pool to reclaim the unspent remainder after a grace window. Funding and signing stay separate roles: the funder owns the pool while a different key signs the vouchers, so a publisher or app can sponsor downloads directly. Its budget pays peer operators per megabyte, and the end user needs no TOKEN, no ETH, and no on-chain transaction — just like a public mirror, except the publisher's budget pays operators directly instead of paying egress to a single cloud. ERC-4337 account abstraction can abstract away gas on top of this.

The peer-mesh model cuts last-mile latency by serving from a nearby operator that already holds the hash, instead of routing every request back to a centralised PoP. Discovery is geographically blind, though, so a hash has no nearby copy until one exists — regional proxy warming is what makes one. Once a local copy exists, selection weighs it against rate and reputation too, so the nearest operator wins only if its price and record hold up as well.

3. Platform

Three roles participate.

Operators are permissionless. Anyone with a machine and a capacity bond can register and serve. The bond is slashable for misbehaviour and unbonds over a 14-day window. Operators set their own per-megabyte rates above a governance-set floor — the only rate bound the protocol enforces; there is no ceiling, and the buyer's protection is that it sees the signed rate before it commits a voucher.

Origins are not permissionless, and onboarding has two steps. Governance vets the publisher wallet once, via a `VETTER_ROLE` holder approving it on-chain — the cold step; once vetted, the publisher seats origin operators for its own namespaces instantly, no further governance needed — the hot step. Either the publisher or governance can unseat an operator. An origin's storage layer stays hidden from the network — the protocol never sees the bucket or file path a blob is served from, only its BLAKE3 hash (and the namespace a request routes on). Hiding the backend stops anyone bypassing the pay-per-byte model by pulling directly from the bucket; vetting the publisher once is what lets the network refuse content without handing any single operator a takedown lever.

Clients are permissionless. A funder-opened `PaymentPool` removes wallet friction for consumer applications: a user can download a file without holding TOKEN, without sending an on-chain transaction, and without knowing a payment pool exists underneath. When a publisher sponsors delivery, that pool is funded from the publisher's budget rather than the user's — the user simply downloads. ERC-4337 account abstraction abstracts gas on top of this.

Compliance enforcement has two points, neither concentrating authority in one party: the publisher vetting gate up front, and an on-chain `ContentBlacklist` that marks hashes non-servable, globally or by region. An operator that keeps serving a blacklisted hash past the compliance window is slashable: anyone can post a challenge bond and submit the operator's own signed response as evidence, and the slash executes on reveal. The blacklist is public and on-chain — enforcement rules are the same for every operator in scope, and every takedown is auditable.

Settlement is self-protecting without a dispute round: an operator redeems the cumulative voucher it holds, and because the funder never submits vouchers on an operator's behalf, closing the pool can't understate what an operator earned. The node binary redeems automatically, with operator-arranged redundancy as a backstop. The owner-close grace window is the censorship defense — sized to at least double the chain's worst-case forced-inclusion delay, so even a censoring sequencer can't lock an operator out of redeeming before the funder reclaims. Clients score candidates on price, latency, and reputation, where reputation is each client's own local record, not a network-wide score. The protocol does not pick winners; it surfaces the signals.

Chain choice is settled, not abstracted: the contracts assume properties of the chain they settle on — most visibly a redemption grace window sized against its worst-case inclusion delay — so porting means matching those properties rather than swapping a backend. The near-term roadmap is production-hardened redemption monitoring, bao verified-range streaming and multi-source parallel fetch through to production, and a production launch fleet — not content encryption, which stays an application concern because the protocol shuttles ciphertext like any other blob.

Network value should scale with operator count: more operators means more places a given hash is already cached, fewer origin pulls, and more geographic coverage. Blobs are fully replicated rather than erasure-coded, so each additional operator is another potential source. Where that curve flattens — the point where new operators buy coverage rather than hit ratio — is a modelling question for the financial model, not something the protocol fixes.

4. Smart Contracts

The on-chain surface is the protocol's trust boundary — anything that doesn't need contract enforcement isn't, and anything on-chain is auditable by anyone with an RPC endpoint. Every contract holding storage, funds, or roles builds on audited OpenZeppelin components — `AccessControl`, `ReentrancyGuard`, a sunsetting `Pausable`, `EIP712` — applied per contract, not uniformly: `ContentBlacklist` skips `Pausable`, since an unlawful-content-removal duty is permanent. No novel cryptography either; the one bespoke piece is the ed25519 verifier, which wraps an audited library routine with the key-decompression and validity guards the EVM's missing native precompile requires. The surface groups into five concerns:

Concern	Contracts
Token & capacity bond	`TOKEN`, `CapacityBond`, `Ed25519Verifier`
Settlement & fees	`PaymentPool`, `FeeRouter`, `BuybackBurner` (venue-neutral base, deployed as a venue-specific subclass)
Slashing & appeals	`SlashJudge`, `SlashAppeal`
Compliance	`ContentBlacklist`, `PublisherRegistry`, `OriginAssignment`, `ManualVettingPolicy`
Governance	`DecdnGovernor`, `TimelockController`

Every governance-tunable economic parameter is bounded at the contract level by hardcoded minimums and maximums. A governance proposal can move a parameter inside its band; nobody can move it past the band without redeploying the contract. (Dependency-address setters are a separate class — they point at other contracts rather than carry a value, and are constrained by the timelock rather than by a numeric band.) The contract-level lower bound on the operator-base `FeeRouter` share is the most load-bearing of these — it is the cashflow invariant that keeps operators receiving enough liquid USDC to cover a meaningful fraction of infrastructure cost regardless of how aggressively governance tunes the other buckets.

5. Token & Governance

deCDN runs a **work-token model**: TOKEN is the asset operators bond in order to serve bytes. Bonding capacity is the only path to protocol rewards and governance weight; passive holders earn nothing. Supply is **fixed at genesis**, with no minting function on the production token contract. Allocation, grouped — the canonical breakdown is eleven categories, summarised here into six, with the marketing and exchange-partnership allocations folded into the totals rather than shown separately:

Allocation	Vesting
Protocol treasury	Partial unlock at genesis, then multi-year linear
Seed & private investors	Multi-year linear with cliff
Core contributors & advisors	Multi-year linear with cliff
App incentives & ecosystem	Partial unlock at genesis, then multi-year linear
Protocol-owned liquidity (Balancer V3 80/20)	Genesis-liquid, held as a governance-controlled LP position
Market making & public sale	Genesis-liquid (the public-sale tranche locks for 6 months if regulatory posture requires)

Genesis liquid supply is a limited initial float. Vesting releases TOKEN unlocked into the recipient's wallet; bonding to operate is a separate, operate-only step. A holder who never runs a node holds liquid TOKEN with no passive-yield path and no governance weight.

Capacity bond. Every operator bonds TOKEN against the bandwidth it declares, on a super-linear curve so high-capacity operators pay more per megabit and the operator set is structurally pushed toward diversity. The capacity bond is the operator's entire TOKEN-side requirement. Bonding is binary: lock to operate, or hold liquid. Each new operator and every

tier upgrade is a fresh buyer of TOKEN, so token demand tracks network capacity growth rather than a promise of return.

Fee split. Each pool redemption's USDC is split atomically by the on-chain `FeeRouter`, all transferred in the settlement transaction. The steady-state target is three buckets:

Bucket	Share (target)	Mechanic
Operator base	60%	Direct per-byte USDC to the operator
Buyback-and-burn	30%	USDC swapped to TOKEN against the Balancer V3 80/20 pool, then burned
Protocol treasury	10%	To the timelock-controlled treasury; funds protocol operations

The network launches with that middle bucket dormant. The genesis deploy ships no buyback contract wired and a 90% operator / 0% buyback / 10% treasury split, so nothing is burned at launch. Governance moves it to the target above in one timelocked call, once the buyback venue is seeded, the burner is wired and audited, and the remaining activation criteria are met.

Activated steady-state target. At launch the buyback bucket is dormant and the split is 90 / 0 / 10.

Once activated, buyback-and-burn is the deflationary prong: routed USDC buys TOKEN on the open pool and burns it, raising demand for every bond-holder uniformly. Operator-aligned value is the same fraction of every settlement either way, but it arrives differently: all cash at launch, base cash plus burn after activation. Node-to-node cache-miss pulls take the same `FeeRouter` split, whichever configuration is live — no bypass.

Each share is governable within hardcoded bounds, and the three must sum to 100%. The bounds cut both ways: the operator base has a 40% floor governance can't remove, keeping operators covering a meaningful fraction of infrastructure cost — and a 90% ceiling, with a 5% floor on an active burn bucket, so operators can't vote the value-accrual mechanism away from non-operator holders either.

Bonding and slashing. Operators bond into `CapacityBond` with a 14-day unbonding window, slashable during unbonding. The slash schedule escalates per lifetime offence (5% / 15% / 50%), with auto-ejection once bond falls below half the operator's tier minimum. Slashed TOKEN is held in escrow until finality, then split **50% to the challenger, 50% burned**. Appeals run through the standalone `SlashAppeal` contract; a successful appeal refunds the full escrowed amount to the operator as their own TOKEN. Restitution is simply the return of escrowed capital, so a wrongly-slashed operator is made whole directly.

Governance. Governance is **operator-only**. Voting weight comes from proven delivered bytes (the `FeeRouter` served-bytes window), scaled by a tenure age-ramp and capped per operator at 5% (itself governable within a band). Fresh bonds vote at zero; full weight needs both sustained

delivery and enough tenure, and a slash zeroes the weight for that window. Non-operator holders — team, investors, treasury, public sale — get no vote unless they also bond and serve bytes, which is what keeps the work-token framing real rather than rhetorical. Proposal threshold, quorum, voting period, and timelock are all bounded, so no vote can move them outside the safe range. `TimelockController` holds `GOVERNANCE_ROLE` from deployment onward, in both phases, putting a 48-hour delay on every change. For the first 6 to 12 months a 5-of-9 bootstrap multisig holds only the right to schedule changes through the timelock (`PROPOSER_ROLE` + `CANCELLER_ROLE`); the transition to operator-weighted proposing is that multisig's own judgment call, not an automated threshold.

6. Discussion

The token layer rewards bonding capacity and serving bytes, not passive holding. Every operator earns the same per-byte USDC base, paid directly and protected by a floor governance can't remove, so commodity operators face no participation gradient and day-to-day P&L stays independent of TOKEN price.

A second deliberate trade-off: operators are permissionless, origins are gated. A delivery network that cannot decline content cannot operate in real jurisdictions. The on-chain blacklist is auditable but visible to adversaries — every takedown is a public event. We accept that visibility in exchange for two enforcement points, neither of which concentrates authority in any single party: governance-gated publisher vetting up front, and on-chain takedown enforcement after the fact.

Adaptive parameter tuning is left to post-launch governance; shipping the launch network does not require it, but shipping a decade-resilient one likely does.

7. Conclusion

deCDN opens the supply side of content delivery. Any operator with bandwidth and a capacity bond can serve. Prices are posted per operator, settled per megabyte in USDC. Origins stay hidden — the protocol never learns the storage location a blob is served from, only its hash — behind a governance-gated publisher-vetting step. The thesis: the underlying market clears at approximately $0.01/GB once the supply side is competitive, well below what incumbent CDNs sell on annual contracts.

Contact: **info@decdn.org**.
